

Constraints Discovery

Khalid Belhajjame

Problem Definition

- Given a dataset, e.g., a CSV file, we would like to discover the constraints that are satisfied by the dataset.
- This help:
 - Profile the dataset,
 - Manage data quality problems that may arise as a result of updates or insertions to the dataset, and also in their repair.
 - Indexing the dataset for efficient access, if it is large.

What Types of Constraints ?

- Key discovery
- Functional Dependency Discovery
- Conditional Functional Dependency Discovery
- Denial Constraint Discovery
- Inclusion Dependency Discovery

There exists some tools that can be used to discover constraints, in particular functional dependencies.

FD Tool

FDTool is a Python command-line application that mines:

- functional dependencies,
- equivalences and
- candidate keys from datasets

Installation:

```
pip install fdtool
```

Run Fdtool:

```
Fdtool path_to_file
```

Github repository: <https://github.com/USEPA/FDTool>

Example using FDTool

```
FDTool-master — -bash — 133x28
[Khalids-MacBook-Pro:FDTool-master khalidbelhajjame$ fdtool data/input/Table1.csv

Reading file:
data/input/Table1.csv

Functional Dependencies:
{A} -> {D}
{D} -> {A}
{A, B} -> {E}
{C, E} -> {A}
{B, E} -> {A}

Equivalences:
{A} <=> {D}
{A, B} <=> {B, E}

Keys:
{B, C, D}
{A, B, C}
{B, C, E}

Time (s): 0.0223
Row count: 7
Attribute count: 5
Number of Equivalences: 2
Number of FDs: 5
Number of FDs checked: 19
Khalids-MacBook-Pro:FDTool-master khalidbelhajjame$
```

FDTool

- Try FDTool on other examples:
 - For example, the examples we've seen in the lecture.
- Beware that if you use a large CSV, you may have to wait several minutes before getting the results

- It is good have tools that are readily available and can be used, but we need to know how they operate
 - In particular, what is the algorithmic solution they implement ?

How Can We Detect Functional Dependencies ?

- Consider a relational schema $R(A,B,C,D)$.
- Try to find a method for identifying all the minimal functional dependencies of R , given a relational instance r of R .

Any thoughts?

FASTFD

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>t</i> ₁	<i>a</i> ₁	<i>b</i> ₁	<i>c</i> ₁	<i>d</i> ₁
<i>t</i> ₂	<i>a</i> ₂	<i>b</i> ₁	<i>c</i> ₁	<i>d</i> ₂
<i>t</i> ₃	<i>a</i> ₁	<i>b</i> ₂	<i>c</i> ₂	<i>d</i> ₁

FASTFD is an instance-based FD discovery algorithm.

- We start by defining the difference set of two tuples *t*₁, *t*₂ as

$$D(t_1, t_2) = \{A \in \text{attr}(R) \mid t_1[A] \neq t_2[A]\}.$$

- The difference sets of a set of tuples *I* are

$$D_I = \{D(t_1, t_2) \mid t_1, t_2 \in I, D(t_1, t_2) \neq \emptyset\}. \quad D = \{AD, ABCD, BC\}$$

- Given a fixed $A \in \text{attr}(R)$, the difference sets of *I* modulo *A* are

$$D_I^A = \{D - \{A\} \mid D \in D_I \text{ and } A \in D\}$$

- An FD $X \rightarrow A$ is a valid FD if and only if *X* covers D_I^A , *i.e.*, *X* intersects with every element in D_I^A .
- The intuition is that if *X* intersects with every element in D_I^A , then *X* distinguishes any two tuples that disagree on *A*.

Example

- Consider a table I of R with four attributes as follows:
- We have
$$D(t_1, t_2) = \{AD\},$$
$$D(t_1, t_3) = \{BC\}, \text{ and}$$
$$D(t_2, t_3) = \{ABCD\}.$$
- Thus, $D_I = \{AD, BC, ABCD\}$,
- FDs of the form $X \rightarrow A$
 - We have $D_I^A = \{D, BCD\}$.
 - Since $\{D\}$ is a minimal cover of D_I^A , we have $D \rightarrow A$.
- Try to find the FDs of the form $X \rightarrow B$, $X \rightarrow C$ and $X \rightarrow D$
 - $C \rightarrow B$
 - $B \rightarrow C$
 - $D \rightarrow A$

	A	B	C	D
t_1	a_1	b_1	c_1	d_1
t_2	a_2	b_1	c_1	d_2
t_3	a_1	b_2	c_2	d_1

FASTFD

- Therefore, the problem of finding all valid FDs is transformed to the problem of finding all minimal set covers of D_I^A for every attribute $A \in \text{attr}(R)$.
- Every subset of $\text{attr}(R) - A$ is a potential candidate minimal cover of D_I^A .

FASTFD Algorithm

Algorithm 2 FASTFD

Input: One relational instance I , schema R

Output: All minimal FDs Σ

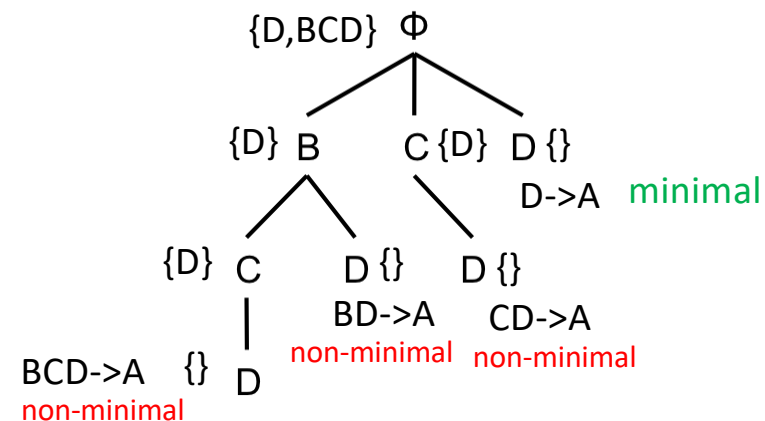
- 1: **for all** $A \in attr(R)$ **do**
 - 2: calculate D_I^A
 - 3: **for all** $A \in attr(R)$ **do**
 - 4: Finding all minimal set covers of D_I^A using a depth-first search
 - 5: For every cover X , add $X \rightarrow A$ to Σ
-

In the following, we describe the depth-first search (Line 4) of Algorithm 2 using the table presented previously

FASTFD Algorithm: Example

- To generate all possible minimal set covers for D_1^A , that is all subsets of $\{BCD\}$, without repetition, the attributes are lexically ordered, i.e., $B > C > D$, and arranged in a depth-first search tree, as shown in the figure.
- A node allow us to obtain a cover if we exhausted all difference sets in D_1^A
 - Not however, that the obtained attributes combinations may not be minimal

	A	B	C	D
t_1	a_1	b_1	c_1	d_1
t_2	a_2	b_1	c_1	d_2
t_3	a_1	b_2	c_2	d_1



- If the algorithm reaches at a node where there are no remaining difference sets left, we have reached a cover X , which may not be minimal. If every immediate subset of X is not a cover, then X is minimal.
- If a node is reached where there are still remaining difference sets, but no attributes left, the depth-first search procedure terminates.

Exercise

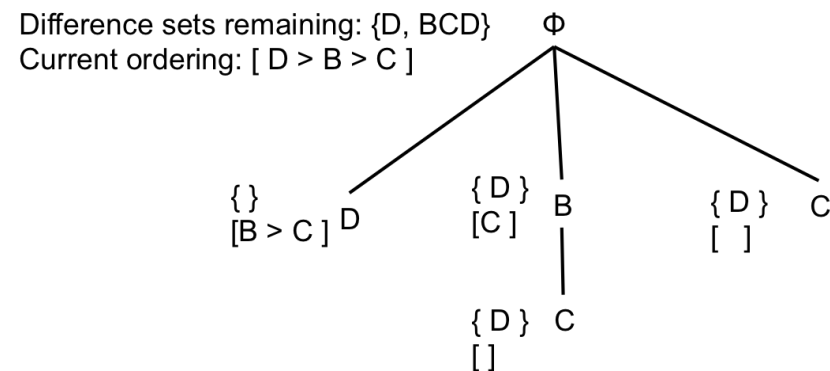
- Run (manually) the FASTFD algorithm to identify the FDs of the following relation

	A	B	C	D
t_1	1	1	1	1
t_2	1	2	2	2
t_3	2	1	2	2
t_4	2	1	1	2

FASTDF Algorithm: Example

- An improved version of the search orders the remaining attributes dynamically according to how many difference sets they cover.
- Ties are broken lexically. For example, to search for minimal covers of DIA using $\{BCD\}$, the attributes are ordered $D > B > C$, since D covers two difference sets, while B and C cover one difference set, as shown in Figure.

	A	B	C	D
t_1	a_1	b_1	c_1	d_1
t_2	a_2	b_1	c_1	d_2
t_3	a_1	b_2	c_2	d_1



FASTFD Algorithm Reference

- More details about the FASTFD algorithm can be found in the following paper:

C. M. Wyss, C. Giannella, and E. L. Robertson. FastFDs: A heuristic- driven, depth-first algorithm for mining functional dependencies from relation instances. In International Conference on Big Data Analytics and Knowledge Discovery, pages 101–110, 2001.

TANE Algorithm for discovering FDs

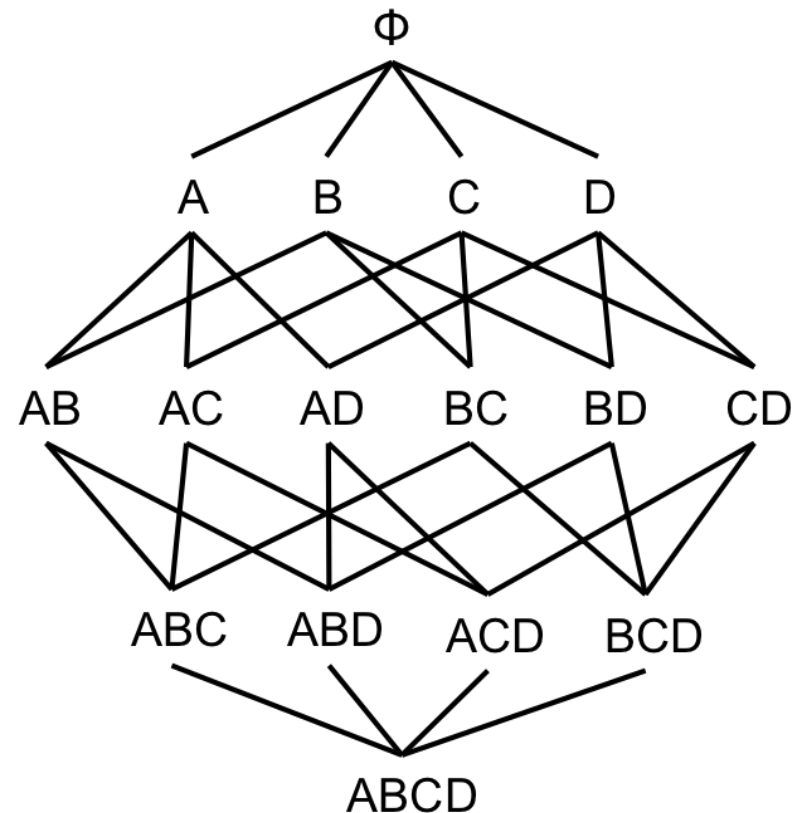
- Assume the relational schema R has m attributes; $|R| = m$.
- Selecting an attribute as the RHS of an FD, any subset of the remaining $m-1$ attributes could serve as the LHS.
 - RHS means Right hand side, and LHS means Left hand side
- Thus, the space to be explored for FD discovery is ?.

TANE Algorithm for discovering FDs

- Assume the relational schema R has m attributes; $|R| = m$.
- Selecting an attribute as the RHS of an FD, any subset of the remaining $m-1$ attributes could serve as the LHS.
- Thus, the space to be explored for FD discovery is $m \times 2^{m-1}$.

TANE Algorithm for discovering FDs

The Figure shows the space of candidate FDs organized in a lattice for a table with four columns, A, B, C, and D, with every edge in the lattice represents a candidate FD. For example, edge A to AC represents the FD $A \rightarrow C$.



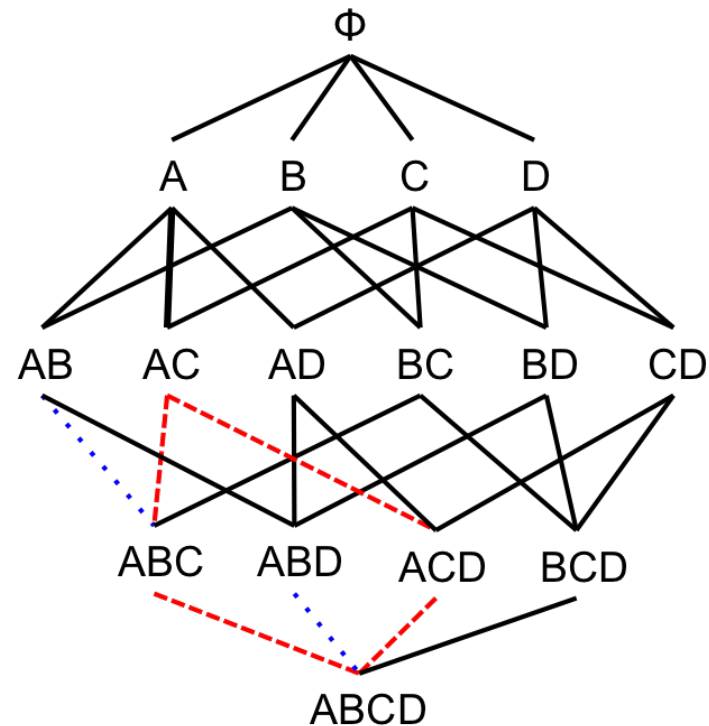
TANE discover minimal FDS

- An FD $X \rightarrow A$ is minimal if for any subset Y of X , the FD $Y \rightarrow A$ does not hold.
- TANE searches the lattice level by level.
- The level-by-level traversal ensures that only minimal FDs are in the output.
- There are three types of pruning employed by TANE:
 1. If $X \rightarrow A \in \Sigma$, then all FDs of the form $XY \rightarrow A$ are implied, and hence they can be pruned;
 2. If $X \rightarrow A \in \Sigma$, then all FDs of the form $XAY \rightarrow B$ can be pruned. The reason is that if $XY \rightarrow B$ is a valid FD, then $XAY \rightarrow B$ is implied by $XY \rightarrow B$, which would be discovered earlier due to level-by-level traversal, and if $XY \rightarrow B$ is not a valid FD, then $XAY \rightarrow B$ is also not valid due to $X \rightarrow A$;
 3. If X is a key, then any node containing X can be pruned.

Example

If $A \rightarrow C$ is valid.

- Using pruning rule 1, we can deduce that:
 - $AB \rightarrow C$ is a valid FD
 - $ABD \rightarrow C$ is a valid FD.
- Using pruning rule 2, we can deduce that the following FDs are not valid:
 - $AC \rightarrow B$
 - $AC \rightarrow D$
 - $ABC \rightarrow D$
 - $ACD \rightarrow B$



TANE algorithm

Algorithm 1 TANE

Input: One relational instance I , schema R

Output: All minimal FDs Σ

```
1:  $L_1 \leftarrow \{\{A\} | A \in attr(R)\}$ 
2:  $l \leftarrow 1$ 
3: while  $L_l \neq \emptyset$  do
4:   for all Node  $Y \in L_l$  do
5:     for all Parent node  $X$  of  $Y$  do
6:       if  $X \rightarrow Y - X$  is valid then
7:         add  $X \rightarrow Y - X$  to  $\Sigma$ 
8:   pruning  $L_l$  based on the three pruning rules
9:    $L_{l+1} \leftarrow$  generate next level based on  $L_l$ 
10:   $l \leftarrow l + 1$ 
```

Exercise

- Build an example of a small relation with few attributes (≤ 4), and small number of rows, that you come up with
- Run (manually) the Tane algorithm on the example that you constructed.

Verifying Functional Dependencies

- A dependency $X \rightarrow A$ holds if all tuples that agree on X also agree on A .
- How can we verify this?

Verifying Functional Dependencies

- A solution that can be used to the verification of dependencies is based on considering sets of tuples that agree on some set of attributes.
- Determining whether a dependency holds or not can be done by checking whether the tuples agree on the right-hand side whenever they agree on the left-hand side.
- This approach is better described using equivalence classes and partitions

Partitions

- Two tuples t and u are *equivalent* with respect to a given set X of attributes if $t[A] = u[A]$ for all A in X .
- Any attribute set X partitions the tuples of the relation into equivalence classes.
- We denote an *equivalence class* of a tuple $t \in r$ with respect to a given set $X \subseteq R$ by $[t]_X$, i.e. $[t]_X = \{u \in r \mid t[A] = u[A] \text{ for all } A \in X\}$.
- The set $\pi_X = \{[t]_X \mid t \in r\}$ of equivalence classes is a *partition* of r under X .
- That is, π_X is a collection of disjoint sets (equivalence classes) of tuples, such that each set has a unique value for the attribute set X and the union of the sets equals the relation r .
- The *rank* $|\pi|$ of a partition π is the number of equivalence classes in π .

Partitions (example)

- Computes the partitions of the attributes A, B, C and D for the following relation.

Tuple ID	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
1	1	a	\$	Flower
2	1	A		Tulip
3	2	A	\$	Daffodil
4	2	A	\$	Flower
5	2	b		Lily
6	3	b	\$	Orchid
7	3	C		Flower
8	3	C	#	Rose

$$\pi_{\{A\}} = \{\{1, 2\}, \{3, 4, 5\}, \{6, 7, 8\}\}$$

$$\pi_{\{B\}} = \{\{1\}, \{2, 3, 4\}, \{5, 6\}, \{7, 8\}\}$$

$$\pi_{\{C\}} = \{\{1, 3, 4, 6\}, \{2, 5, 7\}, \{8\}\}$$

$$\pi_{\{D\}} = \{\{1, 4, 7\}, \{2\}, \{3\}, \{5\}, \{6\}, \{8\}\}$$

Partition Refinement

- The concept of partition refinement gives almost directly functional dependencies.
- A partition π *refines* another partition π' if every equivalence class in π is a subset of some equivalence class of π' .
- Specifically, a functional dependency $X \rightarrow A$ holds if and only if π_X refines $\pi_{\{A\}}$.
- Why is that?

Partition Refinement

- The concept of partition refinement gives almost directly functional dependencies.
- A partition π *refines* another partition π' if every equivalence class in π is a subset of some equivalence class of π' .
- Specifically, **a functional dependency $X \rightarrow A$ holds if and only if π_x refines $\pi_{\{A\}}$.**
- Because if two tuples agree on X then they will definitely agree on A

Partition Refinement

- There is an even simpler test for whether $X \rightarrow A$ holds or not
- ***A functional dependency $X \rightarrow A$ holds if and only if $|\pi_X| = |\pi_{X \cup \{A\}}|$***
- Why is that?

Partition Refinement

- There is an even simpler test for whether $X \rightarrow A$ holds or not
- ***A functional dependency $X \rightarrow A$ holds if and only if $|\pi_X| = |\pi_{X \cup \{A\}}|$***
 - If $X \rightarrow A$, then π_X refines $\pi_{\{A\}}$, which in turns means that $\pi_{X \cup \{A\}}$ equals π_X .
 - On the other hand, since $\pi_{X \cup \{A\}}$ always refines π_X , $\pi_{X \cup \{A\}}$ cannot have the same number of equivalence classes as π_X unless $\pi_{X \cup \{A\}}$ and π_X are equal. That is, $X \rightarrow A$

Computing the Partitions

- The partitions need not be computed from scratch for every set of attributes, rather, TANE computes Π_{XY} from two previously computed partitions, Π_X and Π_Y .
- Note that Π_{XY} contains all subsets of tuples, where each subset is in both Π_X and Π_Y .
- For example, if $\Pi_X = \{\{t1\}, \{t2, t3\}, \{t4\}\}$ and $\Pi_Y = \{\{t1, t2, t3\}, \{t4\}\}$, then
 $\Pi_{XY} = \{\{t1\}, \{t2, t3\}, \{t4\}\}$.
- Therefore, TANE needs only to compute partitions for every single attribute $A \in R$, partitions for every set of attributes X can be computed from a previous level following the level-by-level traversal.

TANE algorithm

More details on the Tane algorithm can be found in this paper:

Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, Hannu Toivonen: TANE: An Efficient Algorithm for Discovering Functional and Approximate Dependencies. Comput. J. 42(2): 100-111 (1999)

Discovering Denial Constraints: FASTDC

- FASTDC is an extension of FASTFD for DC discovery.
- To define the space of DCs, we first need to define the space of predicates P , since DCs are composed of predicates.
- Then, the evidence set Evi_i is built. The evidence set Evi_i is a set, where each element in Evi_i is a subset of predicates in P that are satisfied by a tuple pair in I .
- The evidence set has a similar functionality to the difference set in FASTFD, in that each minimal set cover for Evi_i corresponds to a valid minimal DC.

FASTDC: Example

- Consider the following Employee table with three attributes: Employee ID (I), Manager ID (M), and Salary(S):

TID	I(String)	M(String)	S(Double)
t_9	A1	A1	50
t_{10}	A2	A1	40
t_{11}	A3	A1	40

- For each attribute in the schema, we add two equality predicates ($=, \neq$) between two tuples on it.
- In the same way, for each numerical attribute, we add order predicates ($>, \leq, <, \geq$).
- For every pair of attributes in R, they are joinable (comparable) if equality (order) predicates hold across them, and we add cross column predicates accordingly.

FASTDC: Example

TID	I(String)	M(String)	S(Double)
t_9	A1	A1	50
t_{10}	A2	A1	40
t_{11}	A3	A1	40

- We build the following predicate space P for the table.

$P_1 : t_\alpha.I = t_\beta.I$	$P_5 : t_\alpha.S = t_\beta.S$	$P_9 : t_\alpha.S < t_\beta.S$
$P_2 : t_\alpha.I \neq t_\beta.I$	$P_6 : t_\alpha.S \neq t_\beta.S$	$P_{10} : t_\alpha.S \geq t_\beta.S$
$P_3 : t_\alpha.M = t_\beta.M$	$P_7 : t_\alpha.S > t_\beta.S$	$P_{11} : t_\alpha.I = t_\alpha.M$
$P_4 : t_\alpha.M \neq t_\beta.M$	$P_8 : t_\alpha.S \leq t_\beta.S$	$P_{12} : t_\alpha.I \neq t_\alpha.M$
$P_{13} : t_\alpha.I = t_\beta.M$	$P_{14} : t_\alpha.I \neq t_\beta.M$	

FASTDC: Example

TID	I(String)	M(String)	S(Double)
t_9	A1	A1	50
t_{10}	A2	A1	40
t_{11}	A3	A1	40

$P_1 : t_\alpha.I = t_\beta.I$	$P_5 : t_\alpha.S = t_\beta.S$	$P_9 : t_\alpha.S < t_\beta.S$
$P_2 : t_\alpha.I \neq t_\beta.I$	$P_6 : t_\alpha.S \neq t_\beta.S$	$P_{10} : t_\alpha.S \geq t_\beta.S$
$P_3 : t_\alpha.M = t_\beta.M$	$P_7 : t_\alpha.S > t_\beta.S$	$P_{11} : t_\alpha.I = t_\alpha.M$
$P_4 : t_\alpha.M \neq t_\beta.M$	$P_8 : t_\alpha.S \leq t_\beta.S$	$P_{12} : t_\alpha.I \neq t_\alpha.M$
$P_{13} : t_\alpha.I = t_\beta.M$	$P_{14} : t_\alpha.I \neq t_\beta.M$	

- We build the set of evidence sets:

$\text{Evi}_{\text{Emp}} = \{\{P_2, P_3, P_5, P_8, P_{10}, P_{12}, P_{14}\},$
 $\{P_2, P_3, P_6, P_8, P_9, P_{12}, P_{14}\}, \{P_2, P_3, P_6, P_7, P_{10}, P_{11}, P_{13}\}\}.$

Every element in Evi_{Emp} has at least one pair of tuples in I , such that every predicate in it is satisfied by that pair of tuples.

FASTDC: Example

TID	I(String)	M(String)	S(Double)
t_9	A1	A1	50
t_{10}	A2	A1	40
t_{11}	A3	A1	40

$P_1 : t_\alpha.I = t_\beta.I$	$P_5 : t_\alpha.S = t_\beta.S$	$P_9 : t_\alpha.S < t_\beta.S$
$P_2 : t_\alpha.I \neq t_\beta.I$	$P_6 : t_\alpha.S \neq t_\beta.S$	$P_{10} : t_\alpha.S \geq t_\beta.S$
$P_3 : t_\alpha.M = t_\beta.M$	$P_7 : t_\alpha.S > t_\beta.S$	$P_{11} : t_\alpha.I = t_\alpha.M$
$P_4 : t_\alpha.M \neq t_\beta.M$	$P_8 : t_\alpha.S \leq t_\beta.S$	$P_{12} : t_\alpha.I \neq t_\alpha.M$
$P_{13} : t_\alpha.I = t_\beta.M$	$P_{14} : t_\alpha.I \neq t_\beta.M$	

- We build the set of evidence sets:

$\text{Evi}_{\text{Emp}} = \{\{P_2, P_3, P_5, P_8, P_{10}, P_{12}, P_{14}\},$
 $\{P_2, P_3, P_6, P_8, P_9, P_{12}, P_{14}\}, \{P_2, P_3, P_6, P_7, P_{10}, P_{11}, P_{13}\}\}.$

Every element in Evi_{Emp} has at least one pair of tuples in I, such that every predicate in it is satisfied by that pair of tuples.

- Identify DC as minimal covers

$\mathbf{X}_1 = \{P_2\}$ is a minimal cover, thus $\neg(\overline{P_2})$, i.e., $\neg(t_\alpha.I = t_\beta.I)$ is a valid DC, which states I is a key.

$\mathbf{X}_2 = \{P_{10}, P_{14}\}$ is another minimal cover, thus $\neg(\overline{P_{10}} \wedge \overline{P_{14}})$, i.e., $\neg(t_\alpha.S < t_\beta.S \wedge t_\alpha.I = t_\beta.M)$ is another valid DC, which states that a manager's salary cannot be less than her employee's.

FASTDC

For more information on FASTDC, refer to the following paper.

X. Chu, I. F. Ilyas, and P. Papotti. Discovering denial constraints. Proceedings of the VLDB Endowment, 6(13):1498–1509, 2013.

A more recent approach for discovering Denial Constraints was proposed by:

Tobias Bleifuß, Sebastian Kruse, Felix Naumann: Efficient Denial Constraint Discovery with Hydra. PVLDB 11(3): 311-323 (2017)

Other Algorithms in the literature.

- FASTOFD
- DynFD
- Ainsi que d'autres ...